Unity Portfolio

DunRegey

양지섭



Unity 기술영상 링크

목차

01. 소개

02. UI

03. Player&Monster

04. Dungeon

05. Boss

01-1. 간략 소개

타이틀	DunRegacy
플랫폼	PC
개발환경	Unity 2022.3.21f1 Visual Studio 2022
개발목표	2D 플랫포머 로그라이크
개발기간	7 Weeks
게임목표	어느날 마을에 나타난 던전을 탐험하면서, 던전안에있는 악마를 처단해야합니다.

01-2. Reference

DUNGREED

- 마을에 나타난 던전을 탐색하면서 여러 아이템을 조합해서 층을 클리어하고 보스를 처치하는 게임

- 스프라이트 및 애니메이션, 게임 설정을 참고 하였습니다.



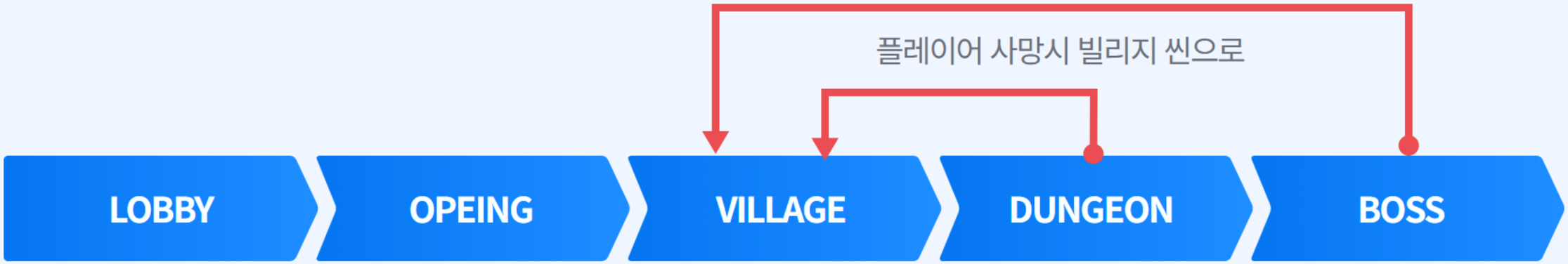
ROGUELEGACY

- 사망시 달라지는 캐릭터및 변화하는 어빌리티로 보스를 처치해야하는 게임

- 게임성 및 게임 진행방향을 참고하였습니다.



01-3. Scene



게임 시작 시 제일 첫 화면
게임시작 및 옵션 화면

기본 조작 및 게임배경
소개

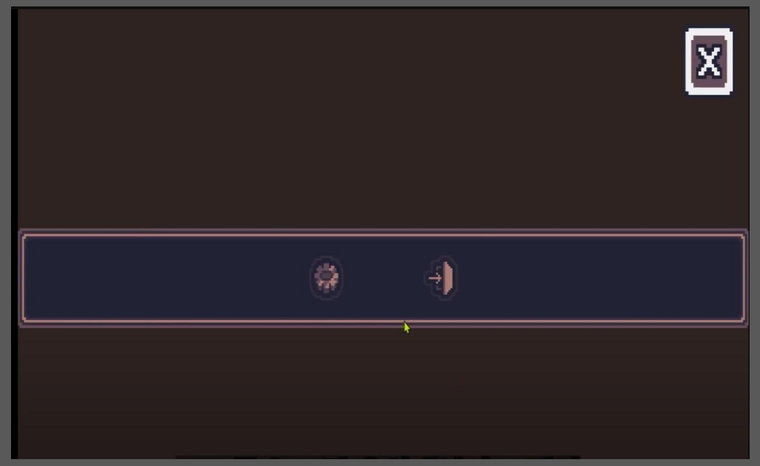
NPC 상호작용 및 어빌
리티 설정 가능

던전 맵 생성 및
몬스터 생성

보스전 및 엔딩씬

- 총 5개의 씬이 존재하고 각씬에 맞는 기능들을 넣었습니다.

02-1. UIManager






```
참조 2개
public void TogglePausePanel()
{
    bool isActive = pausePanel.activeSelf;
    pausePanel.SetActive(!isActive);
    Time.timeScale = isActive ? 1 : 0;  // 게임 일시정지
}

참조 0개
public void ToggleOptionPanel()
{
    bool isActive = optionPanel.activeSelf;
    optionPanel.SetActive(!isActive);
}

참조 1개
public void ToggleExitImage()
{
    bool isActive = exitImage.activeSelf;
    exitImage.SetActive(!isActive);
}

참조 0개
public void LoadLobbyScene()
{
    Time.timeScale = 1;  // 게임 시간 재설정
    TogglePausePanel();
    ToggleExitImage();
    GameManager.Instance.LoadLobbyScene();
}
```

```
// 씬 전환 시 패널 효과
참조 7개
public void StartTransition(System.Action onComplete)
{
    StartCoroutine(FadeIn(onComplete));
}
참조 1개
private IEnumerator FadeIn(System.Action onComplete)
{
    transitionPanel.SetActive(true);
    float duration = 1f;
    float elapsedTime = 0f;

    while (elapsedTime < duration)
    {
        elapsedTime += Time.deltaTime;
        canvasGroup.alpha = Mathf.Lerp(0, 1, elapsedTime / duration);
        yield return null;
    }
    canvasGroup.alpha = 1;

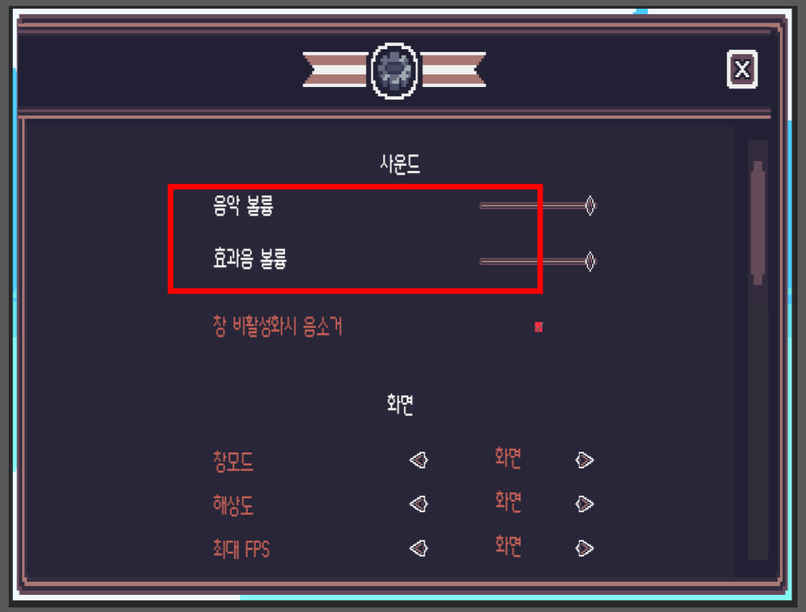
    // 전환 완료 후 콜백 호출
    onComplete?.Invoke();
}

// 씬 전환 후 패널 효과
참조 4개
public void EndTransition()...
참조 1개
private IEnumerator FadeOut()...
```

- UIManager는 각각의 옵션 버튼을 활성화합니다.
- 씬 전환 및 던전의 맵 이동의 경우 자연스러운 페이드 효과를 주기위해서 `StartTransition`메서드를 사용합니다.

02-2. SoundManager

```
public AudioClip HitAudio;
public AudioClip[] boss_Audioclip; // 0 : BGM, 1 : LAUGH , 2 : BULLET, 3 : LASER
public AudioClip[] Player_Audio; // 0.걷는소리 / 1.플레이어가 맞았을때
public AudioClip[] Player_Weapon; // 0.무기 휘두르는 소리
public AudioClip[] UI_Audio; // 0.타운BGM / 1.던전BGM / 2. 죽는소리 / 3.던전입장 / 4.로비 BGM
public AudioClip[] monsterAudio; // 0.스컬독 / 1. 박쥐 / 2.스컬대장
```




```
// 설정창의 볼륨 업데이트
참조 1개
void Volume_Update()
{
    if (bgmSlider != null)
    {
        bgmSave = bgmSlider.value;
        bgmSource.volume = bgmSave;
    }
    if (effectSlider != null)
    {
        effectSave = effectSlider.value;
        effectSource.volume = effectSave;
    }
}
참조 1개
public void Volume_Save()
{
    PlayerPrefs.SetFloat("bgmVolume", bgmSave);
    PlayerPrefs.SetFloat("effectVolume", effectSave);
}
```

```
// BGM 변경
참조 6개
public void PlayBGM(AudioClip newBGM)
{
    if (currentBGM != null && currentBGM == newBGM)
    {
        return;
    }
    StopBGM();
    currentBGM = newBGM;
    bgmSource.clip = newBGM;
    bgmSource.Play();
}
참조 5개
public void StopBGM()
{
    if (bgmSource.isPlaying)
    {
        bgmSource.Stop();
    }
}
```

- `SoundManager`는 각각의 사운드를 배열로 담아서 사용 합니다.
- 언제든지 사운드 매니저에 접근하기 위해서 싱글턴 패턴을 사용 하였습니다.
- 옵션창에서 볼륨조절을 하는 기능도 가지고 있습니다.

02-3. DialogSystem




```
// 배열만큼 반복 실행
참조 1개
private IEnumerator DisplayText()
{
    while (currentDialogIndex < dialogs.Length - 1)
    {
        while (isTypingEffect)
        {
            yield return null;
        }
        SetNextDialog();
    }

    // 마지막 대사를 출력한 후 isTypingEffect가 fals
    while (isTypingEffect)
    {
        yield return null;
    }

    isDialogCompleted = true;
    OnDialogFinished?.Invoke();
}
```

```
// 배열안의 대화랑 화자를 설정함
참조 1개
private void SetNextDialog()
{
    // 이전 화자의 대화 관련 오브젝트 비활성화
    if (currentDialogIndex >= 0)
    {
        SetActiveObjects(speakers[currentSpeakerIndex], false);
    }

    // 다음 대사를 진행하도록
    currentDialogIndex++;

    // 현재 화자 순번 설정
    currentSpeakerIndex = currentDialogIndex % speakers.Length; //

    // 현재 화자의 대화 관련 오브젝트 활성화
    SetActiveObjects(speakers[currentSpeakerIndex], true);

    // 텍스트 타이핑 효과 재생
    typingCoroutine = StartCoroutine(OnTypingText());
}

// 기존 대화 초기화
참조 3개
private void SetActiveObjects(Speaker speaker, bool visible)
{
    speaker.imageDialog.gameObject.SetActive(visible);
    speaker.textDialogue.gameObject.SetActive(visible);
}
```

- Dialog로 게임의 배경설정을 캐릭터의 대화로 보여주게끔 구현했습니다.
- 배열로 넣어놓은 화자와 텍스트가 대화 순서에 따라서 순서대로 구현됩니다.
- 텍스트가 나올때 타이핑 효과도 나오게끔 만들었습니다.
- Dialog 시작 시 뒤에 배경 및 바닥타일도 같이 움직이게끔 구현하였습니다.

```
// 타이핑 효과
참조 1개
private IEnumerator OnTypingText()
{
    int index = 0;
    isTypingEffect = true;

    // 텍스트를 한글자씩 타이핑치듯 재생
    while (index < dialogs[currentDialogIndex].dialogue.Length)
    {
        speakers[currentSpeakerIndex].textDialogue.text = dialogs[
        index++;
        yield return new WaitForSeconds(typingSpeed);
    }

    isTypingEffect = false;
    typingCoroutine = null;
}
```

02-4. Ability



- 게임 시작 시 초기 스크립터블 데이터를 불러옵니다.
- 어빌리티는 매번 10포인트를 사용가능합니다.
- 어빌리티 관련 버튼 클릭 시 스크립터블 데이터에 접근 하여 값을 변경합니다.
- 한 게임에서 저장된 어빌리티는 플레이어 프리팹으로 관리되며 게임 재시작시 초기화 됩니다.

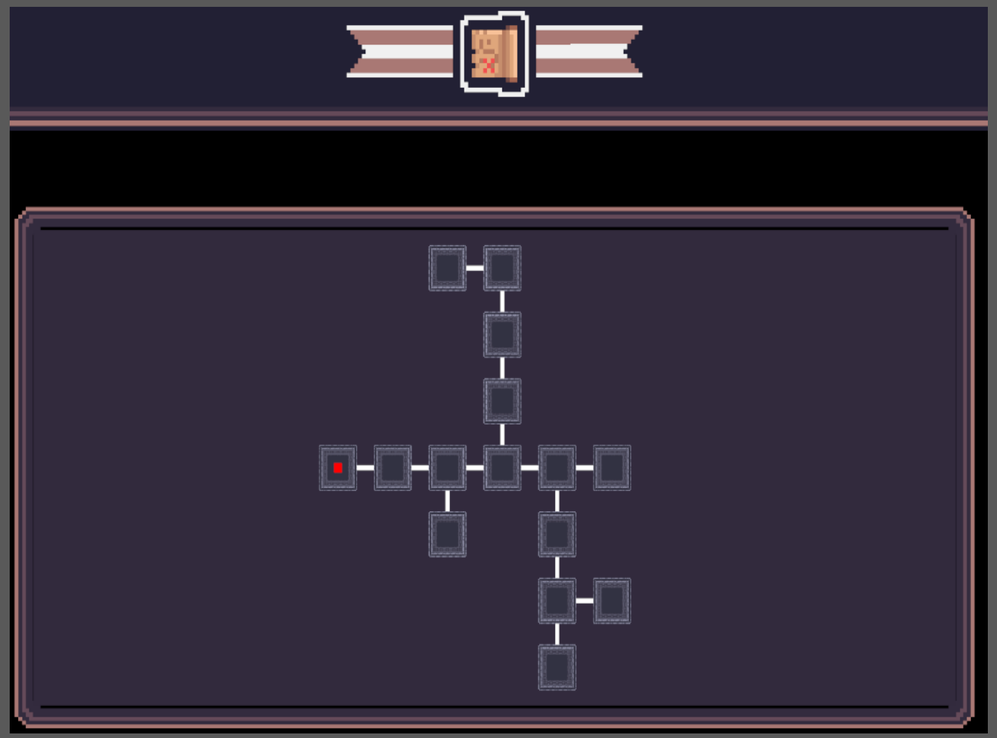
```
public void UpgradeAttackPower(float amount)
{
    playerAttributes.minDamageAmount += amount;
    playerAttributes.maxDamageAmount += amount;
    longSword.weaponData.minDamage = playerAttributes.minDamageAmount;
    longSword.weaponData.maxDamage = playerAttributes.maxDamageAmount;
}

참조 2개
public void UpgradeAttackSpeed(float amount)
{
    playerAttributes.attackSpeed += amount;
    longSword.weaponData.attackSpeed = playerAttributes.attackSpeed;
}

참조 2개
public void UpgradeMoveSpeed(float amount)
{
    playerAttributes.moveSpeed += amount;
    playerController.playerSettings.moveSpeed = playerAttributes.moveSpeed;
}

참조 2개
public void UpgradeMaxHealth(float amount)
{
    playerAttributes.maxHealth += amount;
    playerHealth.maxHealth = (int)playerAttributes.maxHealth;
    playerHealth.currentHealth = playerHealth.maxHealth;
    playerHealth.UpdateHealthUI();
}
```

02-5. MiniMap

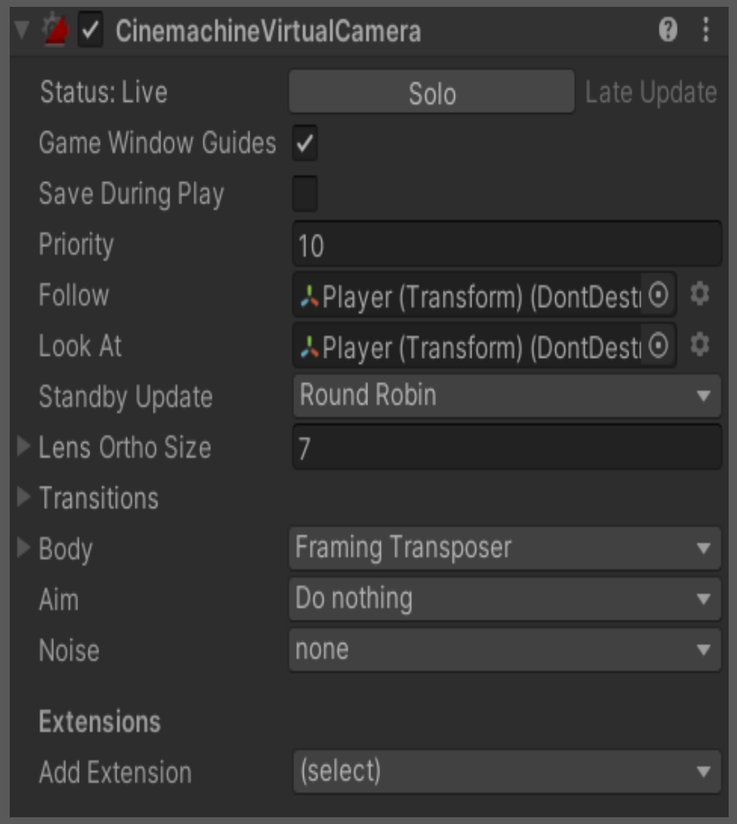


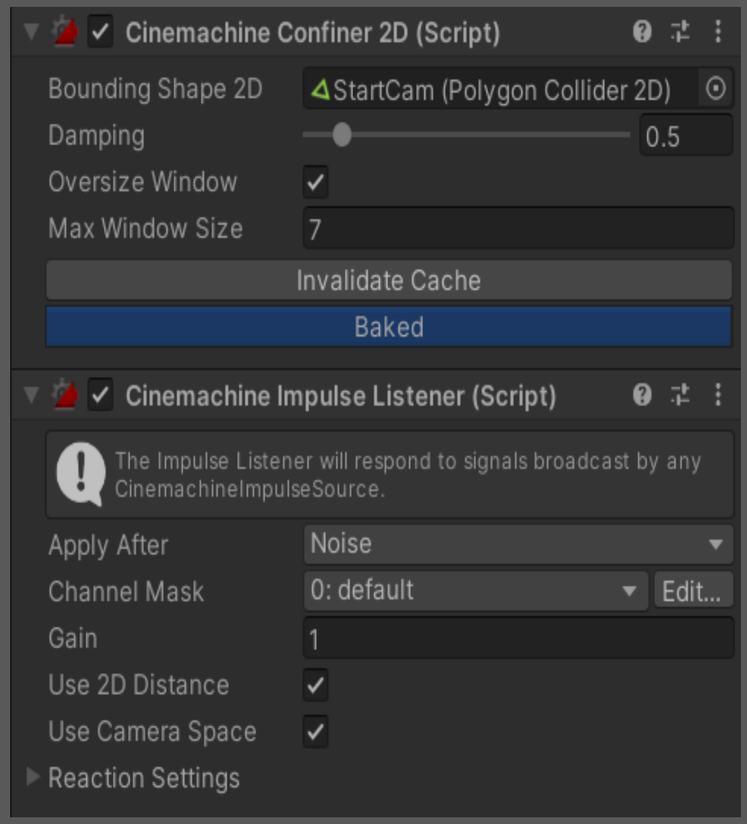


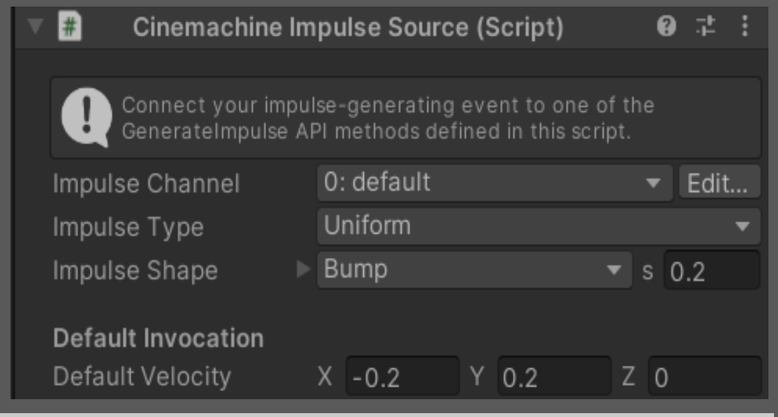


- 미니맵의 경우 던전 방 생성시 미니맵 레이어를 가지고 있는 오브젝트도 같이 생성되게끔 해놨습니다.
- 미니맵 전용 카메라를 만들어서 카메라 레이어를 가지고 있는 오브젝트만 렌더링 하고
  카메라의 출력은 렌더 텍스쳐로 받아와 미니맵 UI에 표시하게끔 구현했습니다.

02-6. CinemachineCamera







- 카메라는 `Cinemachine Camera`를 사용해서 플레이어를 추적하도록 하였습니다.
- `Cinemachine Confiner`를 써서 카메라의 이동을 제한시켜서 맵 밖으로 벗어나는걸 막았습니다.
- ` Cinemachine Impulse`를 사용해서 공격 시 진동효과를 줘서 공격 효과를 내봤습니다.

03-1. Movement

```
// 플레이어 움직임 관련
참조 1개
void Move()
{
    Vector2 moveDirection = new Vector2(PlayerInput.move, playerRigidbody.velocity.y);
    Vector2 moveDistance = moveDirection * playerSettings.moveSpeed;
    playerRigidbody.velocity = new Vector2(moveDistance.x, playerRigidbody.velocity.y);

    if (PlayerInput.move > 0)
        transform.localScale = new Vector2(1, 1);
    else if (PlayerInput.move < 0)
        transform.localScale = new Vector2(-1, 1);

    isWalking = PlayerInput.move != 0;
    playerAnimator.SetFloat("Move", PlayerInput.move);

    dustFX.SetActive(isWalking && isGrounded);

    if (SoundManager.Instance != null && SoundManager.Instance.effectSource != null && SoundManager
    {
        if (isWalking && Time.time >= nextFootstepTime)
        {
            SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.Player_Audio[0]);
            nextFootstepTime = Time.time + footstepDelay;
        }
    }
}
```

```
// 플레이어 점프 관련
참조 1개
void HandleJump()
{
    playerAnimator.SetBool("Jump", true);
    if (Input.GetKeyDown(KeyCode.Space) && isGrounded && playerSettings.jumpCount > 0)
    {
        if (Input.GetKey(KeyCode.DownArrow) || Input.GetKey(KeyCode.S) && currentOneWayPlatfo
        {
            StartCoroutine(DisableCollision());
            return;
        }
        Jump();
        StartCoroutine(JumpCooldown());
    }
    playerAnimator.SetBool("Jump", false);
}
// 아래 점프
참조 1개
private IEnumerator DisableCollision()
{
    if (currentOneWayPlatform != null)
    {
        Collider2D platformCollider = currentOneWayPlatform.GetComponent<Collider2D>();
        if (platformCollider != null)
        {
            Physics2D.IgnoreCollision(playerCollider, platformCollider);
            yield return new WaitForSeconds(0.5f);
            Physics2D.IgnoreCollision(playerCollider, platformCollider, false);
        }
    }
}
// 점프
참조 1개
void Jump()
{
    playerSettings.jumpCount--;
    playerRigidbody.velocity = Vector2.zero;
    playerRigidbody.AddForce(new Vector2(0, playerSettings.jumpForce), ForceMode2D.Impulse);
    jumpFX.SetActive(isGrounded);
}
참조 1개
IEnumerator JumpCooldown()
{
    yield return new WaitUntil(() => isGrounded);
    isGrounded = false;
    playerSettings.jumpCount++;
    yield return new WaitForSeconds(0.5f);
    jumpFX.SetActive(false);
}
```

- 플레이어의 이동의 경우 플레이어 인풋 스크립트에서 입력을 받아서 좌우로 움직이게 구현했습니다.
- 움직임과 동시에 이동효과 및 소리 스프라이트 좌우를 변경 해서 자연스러운 이동을 구현했습니다.
- 점프의 경우 레이캐스트를 플레이어 발 밑에 쏴서 불타입으로 그라운드를 체크하게한다음 땅에있다면 점프가 가능하도록 구현했습니다.
- 아래점프의 경우 특정 플랫폼일시 ` IgnoreCollision`을 사용해서 구현했습니다.

03-2. HealtManager

```
Unity 메시지 | 참조 6개
protected virtual void Awake()
{
    Instance = this;
    currentHealth = maxHealth;
    healthSlider = GetComponentInChildren<Slider>();
    UpdateHealthUI();
    if (HealthManagerInitialized != null)
    {
        HealthManagerInitialized.Invoke();
    }
}

// 피격 시 호출되는 함수
참조 16개
public virtual void OnDamage(float damage)
{
    currentHealth -= (int)damage;

    if (currentHealth <= 0)
    {
        Die();
    }
    healthSlider.value = currentHealth;
}

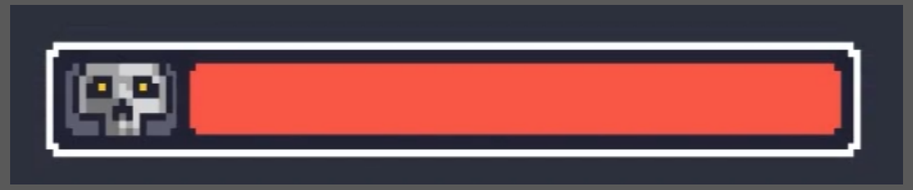
// 체력바 UI 업데이트 함수
참조 4개
public virtual void UpdateHealthUI()
{
    healthSlider.maxValue = maxHealth;
    healthSlider.value = currentHealth;
}

// 사망 처리 함수
참조 6개
protected virtual void Die()
{
    gameObject.SetActive(false);
}
```

```
// 데미지 UI 관련
참조 12개
public override void OnDamage(float damage)
{
    SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.HitAudio);
    float adjustedDamage = damage - monsterData.defense;
    spriteRenderer.color = Color.red;
    base.OnDamage(adjustedDamage);
    spriteRenderer.color = originalColor;
    ShowDamageText((int)adjustedDamage);
}
```

```
// 데미지 관련
참조 10개
public override void OnDamage(float damage)
{
    float adjustedDamage = damage - bossData.defense;
    base.OnDamage(adjustedDamage);
    ShowDamageText((int)adjustedDamage);
}
참조 1개
protected void ShowDamageText(int damage)
{
    GameObject hudTextObject = ObjectPool.Instance.GetDamageTextObject();
    hudTextObject.transform.position = hudPos.transform.position;
    DamageText damageText = hudTextObject.GetComponent<DamageText>();
    if (damageText != null)
    {
        damageText.damage = damage;
        hudTextObject.SetActive(true);
    }
}
```





```
참조 12개
public override void OnDamage(float damage)
{
    if (!isInvincible && !isDead )
    {
        base.OnDamage(damage);
        StartCoroutine(FlashPlayer());
        StartCoroutine(InvincibilityTimer());
    }
}
```



- 체력의 경우 `HealtManager`스크립트를 상속받아 플레이어,몬스터,보스의 체력을 구현했습니다.
- ‘OnDamage`메서드를 상속받아서 각각의 데미지 효과를 다르게 받도록 구현했습니다.
  (ex.플레이어의 경우 일정시간 무적, 몬스터는 플레이어블 데이터의 방어력만큼 감소된 데미지를 받음

03-3. Monster

```
// 몬스터의 상태 열거형
참조 2개
protected virtual void UpdateState()
{
    if (monsterState == MonsterState.Die)
    {
        HandleDieState();
        return;
    }

    switch (monsterState)
    {
        case MonsterState.Idle:
            IdleStand();
            break;
        case MonsterState.Tracking:
            TrackTarget();
            break;
        case MonsterState.Attacking:
            if (!isAttacking)
            {
                StartCoroutine(AttackAfterDelay(monsterData.attackDelay));
            }
            break;
        default:
            break;
    }
}
```

```
// 몬스터의 상태 변수
참조 17개
protected enum MonsterState
{
    Idle,
    Tracking,
    Attacking,
    Die
}
```

```
// 공격 상태 전환
참조 1개
IEnumerator AttackAfterDelay(float delay)
{
    isAttacking = true;
    if (isAttacking)
    {
        if (scopeOfAttackCollider != null)
        {
            monsterRigidbody.velocity = new Vector2(0, monsterRigidbody.velocity.y);

            yield return new WaitForSeconds(delay);

            ExecuteAttackPattern();
            yield return new WaitForSeconds(delay);
            isAttacking = false;
        }
        else
        {
            isAttacking = false;
        }
    }
    monsterState = MonsterState.Idle;
}

// 상속받은 몬스터들의 공격 패턴
참조 5개
protected virtual void ExecuteAttackPattern()
{

}
```

```
protected override void ExecuteAttackPattern()
{
    monsterRigidbody.velocity = Vector2.zero;
    Vector2 targetPosition = target.position;

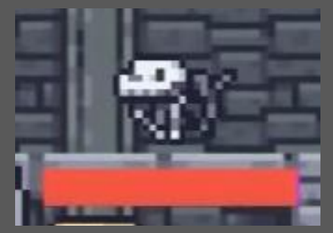
    GameObject bulletObject = ObjectPool.Instance.GetBulletObject();
    if (bulletObject != null)
    {
        SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.monsterAudio[1]);
        bulletObject.transform.position = transform.position;
        bulletObject.SetActive(true);
        Bullet bullet = bulletObject.GetComponent<Bullet>();
        if (bullet != null)
        {
            bullet.Fire(targetPosition, monsterData.damage, monsterData.attackSpeed);
        }
    }
}
```

```
참조 2개
protected override void ExecuteAttackPattern()
{
    monsterAnimator.SetTrigger("Attack");
    SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.monsterAudio[2]);
    Collider2D[] hitPlayers = Physics2D.OverlapBoxAll(transform.position, monsterData.scopeO
    foreach (var hitPlayer in hitPlayers)
    {
        IDamageable damageable = hitPlayer.GetComponent<IDamageable>();
        if (damageable != null)
        {
            damageable.OnDamage(monsterData.damage);
        }
    }
}
```

```
참조 2개
protected override void ExecuteAttackPattern()
{
    isRushing = true;
    hasDamaged = false;
    startPosition = transform.position;
    Vector2 rushForce = (spriteRenderer.flipX ? Vector2.left : Vector2.right) * monsterDat
    monsterRigidbody.AddForce(rushForce, ForceMode2D.Impulse);
    SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.monsterAudio[0]);
}
```







- 몬스터의 경우 현재 상태를 나타내는 `enum`을 만든 후에 스위치문을써서 상태에 따라 맞는 행동을 하도록 하였습니다. (FSM 패턴)
- 몬스터는 총 3개의 패턴으로 나누었고 몬스터 스크립트를 상속받아 공격을 시작할경우 각각의 맞는 패턴이 나오도록 하였습니다.
- 1. 원거리 공격 2. 근접 공격 3. 돌진 공격

04-1. RoomManager / Room




```
// 초기 방 생성 시작
참조 2개
private void StartRoomGenerationFromRoom(Vector2Int roomIndex)
{
    roomQueue.Enqueue(roomIndex); // 큐에 데이터 집어넣기
    int x = roomIndex.x;
    int y = roomIndex.y;
    roomGrid[x, y] = 1;
    roomCount++;

    var initialRoom = Instantiate(startroomPrefab, GetPositionFromGridIndex(roomIndex), Quaternion.identity);

    initialRoom.name = $"Room-{roomCount}";
    initialRoom.GetComponent<Room>().RoomIndex = roomIndex;

    roomObjects.Add(initialRoom);

    currentRoom = initialRoom.GetComponent<Room>(); // 초기 방을 현재 방으로 설정

    CollriderChange();
}
```

```
private void GenerateSecondRoom(Vector2Int roomIndex)
{
    roomQueue.Enqueue(roomIndex);
    int x = roomIndex.x;
    int y = roomIndex.y;
    roomGrid[x, y] = 1;
    roomCount++;
    int randomnumber = Random.Range(0, roomPrefab.Length);

    var secondRoom = Instantiate(roomPrefab[randomnumber], GetPositionFromGridIndex(roomIndex), Quaternion.identity);


    secondRoom.name = $"Room-{roomCount}";
    secondRoom.GetComponent<Room>().RoomIndex = roomIndex;

    roomObjects.Add(secondRoom);

    var roomScript = secondRoom.GetComponent<Room>();
    roomScript.OpenDoor(Vector2Int.left);

    // 인접한 방이 있을 때만 다른 문 열기
    if (x + 1 < gridSizeX && roomGrid[x + 1, y] == 1) roomScript.OpenDoor(Vector2Int.right);
    if (y + 1 < gridSizeY && roomGrid[x, y + 1] == 1) roomScript.OpenDoor(Vector2Int.up);
    if (y - 1 >= 0 && roomGrid[x, y - 1] == 1) roomScript.OpenDoor(Vector2Int.down);
}
```

- 던전생성은 룸 스크립트를 가지고 있는 프리팹들을 룸매니저의 배열에 등록해서 지정된 방의 숫자만큼 첫 방과 마지막 방을 제외한 방들이 생성됩니다.
- 그리드의 크기에 따라서 던전 생성 영역이 설정이 되고 초기 방을 생성하고 큐에 추가합니다.
- 그 이후에 로직에 따라서 방의 위치를 결정하고 방을 생성합니다. 추가적으로 방이 설정한 개수만큼 생성되지않을경우 다시 생성합니다.
- 모든 방이 생성이 되면 방들을 잇는 문들을 열고 나머지는 닫아서 던전생성이 완료됩니다.

04-1. RoomManager / Room

```
[SerializeField] GameObject startroomPrefab;
[SerializeField] GameObject[] roomPrefab;
[SerializeField] GameObject endroomPrefab;
[SerializeField] GameObject miniMapPrefab;

[Header("룸 생성 개수 조절")]
[SerializeField] private int maxRooms = 10;
[SerializeField] private int minRooms = 10;

[Header("룸 영역 조절")]
[SerializeField] int roomWidth = 100;
[SerializeField] int roomHeight = 100;

[Header("그리드 영역 조절")]
[SerializeField] int gridSizeX = 7;
[SerializeField] int gridSizeY = 7;
```

```
// 방 생성 시도
참조 4개
private bool TryGenerateRoom(Vector2Int roomIndex)
{
    int x = roomIndex.x;
    int y = roomIndex.y;

    // 유효성 검사
    if (x < 0 || x >= gridSizeX || y < 0 || y >= gridSizeY)
    {
        return false;
    }

    // Room-1과 Room-2방의 같은 X축에 배치되지 않도록 함
    if (x == 0 || x == 1)
    {
        return false;
    }

    if (roomCount >= maxRooms)
    {
        return false;
    }
    if (Random.value < 0.5f && roomIndex != Vector2Int.zero)
    {
        return false;
    }

    if (CountAdjacentRooms(roomIndex) > 1)
        return false;

    roomQueue.Enqueue(roomIndex);
    roomGrid[x, y] = 1;
    roomCount++;
    int randomnumber = Random.Range(0, roomPrefab.Length);
    GameObject roomToInstantiate = roomCount == maxRooms ? endroo

    var newRoom = Instantiate(roomToInstantiate, GetPositionFromG

    newRoom.GetComponent<Room>().RoomIndex = roomIndex;
    newRoom.name = $"Room-{roomCount}";
    roomObjects.Add(newRoom);

    OpenDoors(newRoom, x, y);

    return true;
}
```

```
// 배열에있는 문들의 콜라이더를 담고 타일맵에서 문들의 콜라이더만큼의 영역을 None으
참조 0개
public void InitializeDoors()
{
    boxColliders = new List<BoxCollider2D>();

    foreach (GameObject door in doors)
    {
        boxColliders.Add(door.GetComponent<BoxCollider2D>());
    }
    DisableTilemapColliderWithinBox();
}
참조 1개
private void DisableTilemapColliderWithinBox()
{
    BoundsInt bounds = tilemap.cellBounds;
    for (int x = bounds.xMin; x < bounds.xMax; x++)
    {
        for (int y = bounds.yMin; y < bounds.yMax; y++)
        {
            Vector3Int cellPosition = new Vector3Int(x, y, 0);
            Vector3 cellWorldPosition = tilemap.CellToWorld(cellPosition);

            foreach (BoxCollider2D boxCollider in boxColliders)
            {
                if (boxCollider.bounds.Contains(cellWorldPosition))
                {
                    tilemap.SetColliderType(cellPosition, Tile.ColliderType.None);
                }
            }
        }
    }
}
```

```
}
// 방 전환 메서드
참조 1개
private IEnumerator HandleDoorCollision(Collider2D collision)
{
    bool transitionCompleted = false;

    UIManager.Instance.StartTransition(() =>
    {
        transitionCompleted = true;
    });

    while (!transitionCompleted)
    {
        yield return null;
    }

    // 충돌한 문이 방의 어느곳인지 저장하고 이동 메서드 작동
    Collider2D[] childColliders = playerCollider.GetComponentsInChildren<Collider2D>(
    foreach (Collider2D childCollider in childColliders)
    {
        Physics2D.IgnoreCollision(playerCollider, childCollider);
    }

    Room currentRoom = roomManager != null ? roomManager.currentRoom : null;

    Vector2Int direction = Vector2Int.zero;

    foreach (Vector2Int dir in new Vector2Int[] { Vector2Int.up, Vector2Int.down, Vec
    {
        if (collision.transform.parent.gameObject == currentRoom.GetDoor(dir)?.gameOb
        {
            direction = dir;
            if (currentRoom.gameObject.name == "Room-1")
            {
                direction = Vector2Int.right;
            }
            break;
        }
    }

    if (direction != Vector2Int.zero)
    {
        MoveToAdjacentRoom(direction);
    }

    yield return new WaitForSeconds(0.5f);

    UIManager.Instance.EndTransition();
```

- 큐를 사용한 이유는 방 생성 순서를 관리하여 던전 생성의 복잡도를 줄이고싶었습니다.
- FiFo 원칙을 이용하여 생성된 방을 순차적으로 처리하게끔 하였습니다.
- 방의 이동의 경우에는 다음 방의 문을 확인후에 열려있는문이 있다면 이동처리가 되게끔 하였습니다.

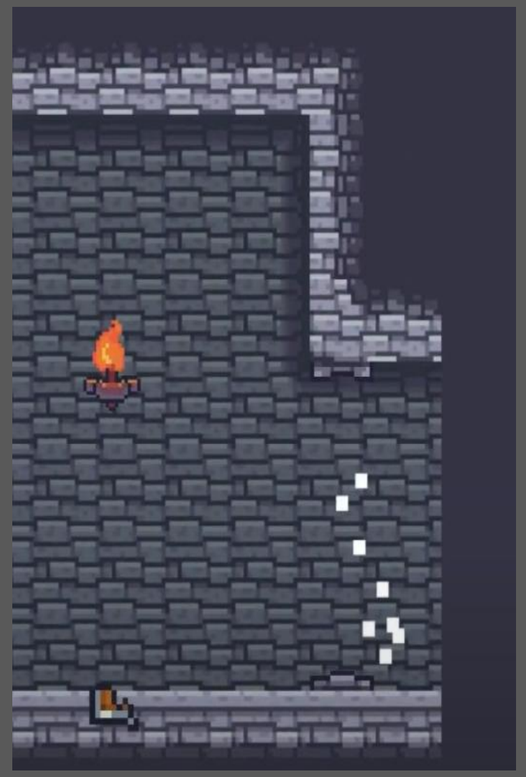
04-2. DungeonManager

```
private void SpawnMonsters()
{
    // 스폰 포인트에서 몬스터 스폰
    for (int i = 0; i < spawnerPoint.Count; i++)
    {
        int randomIndex = Random.Range(0, ObjectPool.Instance.monsterPrefabs.Length);
        GameObject monster = ObjectPool.Instance.GetMonsterObject(randomIndex);
        monster.transform.position = spawnerPoint[i].position;
        monster.SetActive(true);
        spawnedMonsters.Add(monster);
    }
}

참조 1개
private void CheckMonsters()
{
    // 모든 몬스터가 제거되었는지 확인
    bool allMonstersCleared = true;
    foreach (GameObject monster in spawnedMonsters)
    {
        if (monster.activeSelf)
        {
            allMonstersCleared = false;
            break;
        }
    }

    // 모든 몬스터가 제거되었으면 문을 열기
    if (allMonstersCleared)
    {
        isClear = true;
    }
}
```





- 던전 매니저는 각 방들의 프리팹마다 가지고 있고 지정되어있는 몬스터 스폰 포인트에서 랜덤한 몬스터를 스폰합니다.
- 플레이어가 모든 몬스터를 잡았을경우 방들의 문이 열리고 다음방으로 이동이 가능합니다.
- 기존에 클리어한 방은 다시 들어와도 몬스터가 스폰되지 않게끔 설정해놨습니다.

05-1. BossManager

```
// 보스 등장시 이펙트 효과
참조 1개
public void Screen_Effect()
{
    if (!bossStart)
        return;

    if (!firstCheck)
    {
        SoundManager.Instance.bgmSource.PlayOneShot(SoundManager.Instance.boss_Audioclip[0]);
        firstCheck = true;
        bossSceneStart = true;

        cinemachineCamera.Follow = bossTransform;
        cinemachineCamera.LookAt = bossTransform;
    }
    // 상하 패널 효과
    foreach (Image img in ScreenBox)
    {
        Color alpha = img.color;
        alpha.a += Time.deltaTime * 1f;
        img.color = alpha;

        if (img.color.a >= 1)
        {
            bossStart = false;
        }
    }

    foreach (SpriteRenderer sprite in bossSpriteRenderers)
    {
        Color alpha = sprite.color;
        alpha.a += Time.deltaTime * 1f;
        sprite.color = alpha;
    }


    if (!bossStart)
    {
        SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.boss_Audioclip[1]);
        StartCoroutine(Boss_Name_Coroutine());
    }
}
```

```
// 보스 사망시 스크린 효과
참조 1개
public IEnumerator End_Screen_Coroutine()
{
    Color al = End_Screen.color;
    al.a = 1;
    End_Screen.color = al;

    while (true)
    {
        Color alpha = End_Screen.color;
        alpha.a -= Time.deltaTime * 0.5f;
        End_Screen.color = alpha;

        if (End_Screen.color.a <= 0)
        {
            break;
        }
        yield return null;
    }
}

참조 1개
public void TriggerOnBossDeath()
{
    if (OnBossDeath != null)
    {
        OnBossDeath.Invoke();
    }
}

참조 1개
public void OnBossDeathHandler()
{
    Animator targetAnimator = clearSealStone.GetComponent<Animator>();
    Collider2D targetCollider = clearSealStone.GetComponent<Collider2D>();

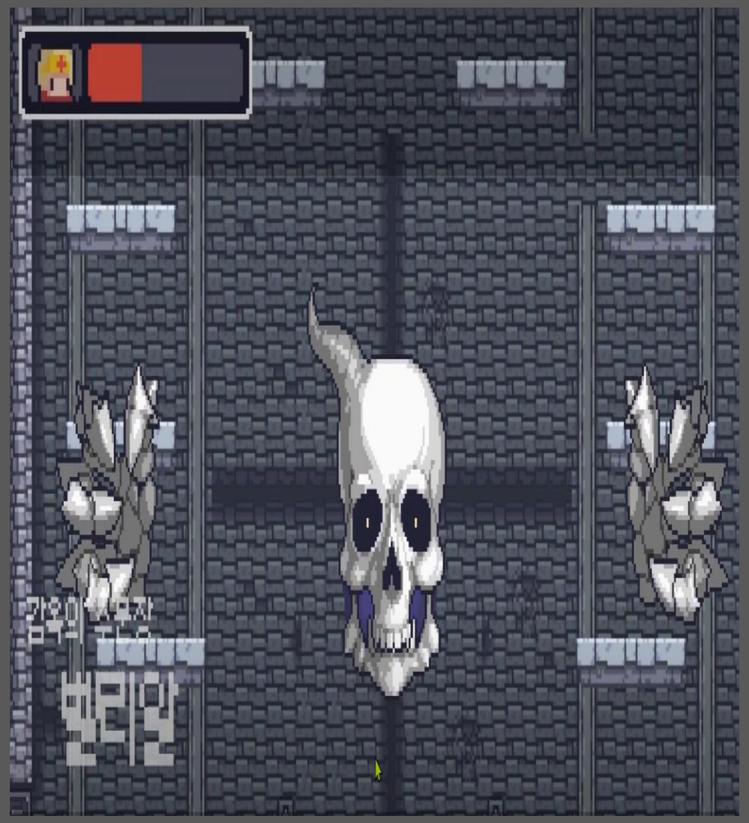
    if (targetAnimator != null)
    {
        targetAnimator.SetTrigger("SealOpen");
    }
    if (targetCollider != null)
    {
        targetCollider.isTrigger = true;
    }
}
```

```
public Canvas bossCanvas;

public bool firstCheck = false;
public bool bossStart = false;
public bool bossSceneStart = false;
public int name_Count = 0;

[SerializeField]
private Image[] ScreenBox;
[SerializeField]
private SpriteRenderer[] bossSpriteRenderers;
[SerializeField]
private Text[] BossName_Text;
[SerializeField]
private Image End_Screen;
[SerializeField]
private CinemachineVirtualCamera cinemachineCamera;
[SerializeField]
private Transform playerTransform;
[SerializeField]
private Transform bossTransform;

public event Action OnBossDeath;
public GameObject clearSealStone;
```



- 보스 매니저는 보스 씬 입장 시 특정위치에 오면 등장 효과와 함께 보스전의 시작을알립니다.
- 패널과 효과들의 알파값으로 효과가 나타나고 코루틴으로 호출하게 되어있습니다.
- 추가적으로 보스 사망시 사망 패널 효과와 함께 다음 던전으로 가는 방의 문이 열리게끔 해놨습니다.

05-2. Boss

```
참조 5개
public IEnumerator PatternCoroutine()
{
    yield return new WaitForSeconds(2f);

    if (currentHealth > 0)
    {
        bossPattern = Random.Range(1, 5);
        Attack();
    }
}
참조 1개
private void Attack()
{
    switch (bossPattern)
    {
        case 1:
            int randDir = Random.Range(0, 2);
            ShootLaser(randDir);
            StartCoroutine(PatternCoroutine());
            break;
        case 2:
            StartCoroutine(TripleLaser());
            break;
        case 3:
            StartCoroutine(CreateSwordCoroutine(0));
            break;
        case 4:
            StartCoroutine(ShootBulletCoroutine());
            break;
    }
}
```

```
참조 3개
protected override void Die()
{
    GameManager.Instance.isDungeonEntryComplete = true;
    SoundManager.Instance.StopBGM();
    StartCoroutine(DieCoroutine());
}
참조 1개
IEnumerator DieCoroutine()
{
    GameManager.Instance.SetPlayerInputActive(false);
    Time.timeScale = 0.5f;
    SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.boss_Audioclip[4]);
    StartCoroutine(BossManager.Instance.End_Screen_Coroutine());

    yield return new WaitForSeconds(1f);
    Time.timeScale = 1f;

    int Count = 0;
    int lastCount = 1;

    while (true)
    {
        for (int i = 0; i < 10; i++)
        {
            float randPosX = Random.Range(-5f, 5f);
            float randPosY = Random.Range(-4f, 4f);

            GameObject obj = Instantiate(dieEffectPrefab);

            obj.transform.position = new Vector3(transform.position.x + randPosX,
                transform.position.y + randPosY, 0);

            SoundManager.Instance.effectSource.PlayOneShot(SoundManager.Instance.boss_Audioclip[5]);

            StartCoroutine(DieEffectOffCoroutine(obj));
        }
        yield return new WaitForSeconds(0.3f);

        if (++Count > 15)
            break;
    }

    foreach (GameObject obj in bossOtherObj)
```







- 보스의 경우 총 4가지 패턴으로 구상했습니다.
- 1. 레이저 발사 2. 레이저 3연속 발사 3.검 날리기 4. 나선형 구체 발사
- 코루틴으로 대기시간을 두고 패턴 발동 시 랜덤한 수를 받아서 스위치문으로 패턴이 나오게 구성했습니다.
- 보스 사망시 사운드 효과와 사망 효과를 추가하였습니다.

Thanks for watching!

양지섭

Copyright ⓒ mangoboard All Rights Reserved.